

Project Documentation: Serial-In Parity Generator Unit

Digital Design Lab

Project Overview

This lab implements a parameterized **Serial Parity Generator** module in Verilog HDL[cite: 31]. The circuit accepts a serial data bitstream via `serialin` and shifts incoming bits into an internal register (`shift_reg`) on each rising clock edge[cite: 31]. A Verilog function (`calcparity`) evaluates parity across the registered parallel frame iteratively using a `for` loop[cite: 31].

Specifications & Functionality

- **Register Width (w):** Parameterized to 8 bits by default[cite: 31, 32].
- **Synchronous Reset:** Active-high reset clears internal register state to zero on `posedge clk`[cite: 31].
- **Serial Shifting:** Performs left-shift operation on `posedge clk`, appending incoming `serialin` to LSB[cite: 31].
- **Iterative Parity Calculation:** Computes parity using an explicit loop iterating across all `w` bits of `shift_reg`[cite: 31].

Port Summary

Port Name	Direction	Type	Width	Description
clk	Input	wire	1	Clock input signal[cite: 31, 32]
rst	Input	wire	1	Synchronous active-high reset[cite: 31, 32]
serialin	Input	wire	1	Serial data input line[cite: 31, 32]
parity_out	Output	wire	1	Calculated parity bit output[cite: 31, 32]

Table 1: Parity Generator Module Port Interfaces

Verilog Source Code

Design Module (parity_out.v)

```
module parity_out #(
    parameter w = 8
)(
    input  wire clk,
    input  wire rst,
    input  wire serialin,
    output wire parity_out
);
```

```

reg [w-1:0] shift_reg;

// Iterative function calculating parity across array width
function calcparity;
    input [w-1:0] datain;
    integer i;
    begin
        calcparity = 1'b0;
        for (i = 0; i < w; i = i + 1) begin
            calcparity = calcparity ^ datain[i];
        end
    end
endfunction

// Synchronous shift process
always @(posedge clk) begin
    if (rst)
        shift_reg <= {w{1'b0}};
    else
        shift_reg <= {shift_reg[w-2:0], serialin};
end

// Continuous assignment calling calcparity function
assign parity_out = calcparity(shift_reg);

endmodule

```

Testbench Module (tb_parity_out.v)

```

`timescale 1ns / 1ps

module tb_parity_out;

    parameter w = 8;

    reg clk;
    reg rst;
    reg serialin;

    wire parity_out;

    parity_out #(
        .w(w)
    ) uut (
        .clk(clk),
        .rst(rst),
        .serialin(serialin),
        .parity_out(parity_out)
    );

    always #5 clk = ~clk;

    reg [w-1:0] test_data = 8'b10110010;
    integer i;

```

```

initial begin
    clk = 0;
    rst = 1;
    serialin = 0;

    #20;
    rst = 0;

    for (i = 0; i < w; i = i + 1) begin
        @(negedge clk);
        serialin = test_data[i];

        @(posedge clk);
        #1;
        $display("Time = %0t | Shift Reg = %b | Input = %b | Parity
                  Out = %b",
                  $time, uut.shift_reg, serialin, parity_out);
    end

    #20;
    $finish;
end

endmodule

```

Simulation & Verification

Execution Command

To run batch simulation using Cadence Xcelium or ModelSim:

```
xrun parity_out.v tb_parity_out.v
```

Expected Simulation Output

The simulation tracks state progression step-by-step as bits are streamed into the register:

Time = 26000		Shift Reg = 00000000		Input = 0		Parity Out = 0
Time = 36000		Shift Reg = 00000001		Input = 1		Parity Out = 1
Time = 46000		Shift Reg = 00000010		Input = 0		Parity Out = 1
Time = 56000		Shift Reg = 00000100		Input = 0		Parity Out = 1
Time = 66000		Shift Reg = 00001001		Input = 1		Parity Out = 0
Time = 76000		Shift Reg = 00010011		Input = 1		Parity Out = 1
Time = 86000		Shift Reg = 00100110		Input = 0		Parity Out = 1
Time = 96000		Shift Reg = 01001101		Input = 1		Parity Out = 0